

@RequestBody, @PathVariable, @RequestParam

These three Spring MVC annotations are used to extract **client input from an HTTP request**. For a 3-year Spring Boot interview, you should know not only the syntax but **when to use each one and why**.

1. @RequestBody

@RequestBody is used when the client sends data in the **HTTP request body**, usually as JSON.

Example

```
POST /api/v1/payments
Content-Type: application/json
```

```
{
  "customerId": "C101",
  "amount": 1500,
  "currency": "INR"
}
```

Controller:

```
@PostMapping("/payments")
public PaymentResponse createPayment(
    @RequestBody PaymentRequest request) {

    return paymentService.createPayment(request);
}
```

Spring uses **Jackson** to convert the JSON into the Java object.

```
JSON
↓
Jackson
↓
PaymentRequest object
↓
Controller
```

DTO

```
public class PaymentRequest {

    private String customerId;
    private BigDecimal amount;
    private String currency;

    // getters/setters
}
```

With validation

Very common in real applications:

```
@PostMapping("/payments")
public PaymentResponse createPayment(
    @Valid @RequestBody PaymentRequest request) {

    return paymentService.createPayment(request);
}
```

Now annotations such as:

```
@NotBlank
@NotNull
@Positive
```

can validate the request.

When to use it

Use `@RequestBody` when you have a **structured payload**, especially for:

- POST
- PUT
- PATCH

2. @PathVariable

`@PathVariable` extracts a value directly from the **URL path**.

Example:

```
GET /api/v1/payments/12345
```

Here:

```
12345
```

is part of the resource URL.

Controller:

```
@GetMapping("/payments/{paymentId}")
public PaymentResponse getPayment(
    @PathVariable String paymentId) {

    return paymentService.getPayment(paymentId);
}
```

The `{paymentId}` in the mapping is connected to:

```
@PathVariable String paymentId
```

Why use @PathVariable?

Use it when the value identifies a **specific resource**.

Examples:

```
GET /payments/123
GET /customers/456
GET /orders/789
```

Think:

Which resource?

```
/payments/123
      ↑
    PathVariable
```

3. @RequestParam

`@RequestParam` extracts values from the **query string**.

Example:

```
GET /api/v1/payments?status=SUCCESS
```

Controller:

```
@GetMapping("/payments")
public List<PaymentResponse> getPayments(
    @RequestParam String status) {

    return paymentService.getPayments(status);
}
```

Here:

```
?status=SUCCESS
```

contains a query parameter.

Multiple request parameters

```
GET /payments?status=SUCCESS&currency=INR
```

```
@GetMapping("/payments")
public List<PaymentResponse> getPayments(
    @RequestParam String status,
    @RequestParam String currency) {

    return paymentService.getPayments(status, currency);
}
```

Optional request parameter

```

@GetMapping("/payments")
public List<PaymentResponse> getPayments(
    @RequestParam(required = false) String status) {

    ...
}

```

Now both are allowed:

```
GET /payments
```

and

```
GET /payments?status=SUCCESS
```

You can also provide a default:

```

@RequestParam(defaultValue = "SUCCESS")
String status

```

4. The easiest way to remember

Suppose your BPS application has this API:

```
GET /api/v1/customers/C101/payments/12345?status=SUCCESS
```

Break it down:

```

/api/v1/customers/
    C101
    ↑
    @PathVariable

/payments/
    12345
    ↑
    @PathVariable

?status=SUCCESS
    ↑
    @RequestParam

```

Example:

```

@GetMapping("/customers/{customerId}/payments/{paymentId}")
public PaymentResponse getPayment(
    @PathVariable String customerId,
    @PathVariable String paymentId,
    @RequestParam(required = false) String status) {

    ...
}

```

5. @RequestBody vs @PathVariable vs

@RequestParam

Annotation	Comes from	Typical use
@RequestBody	HTTP body	Create/update structured data
@PathVariable	URL path	Identify a specific resource
@RequestParam	Query string	Filtering, searching, pagination, optional parameters

Example:

```
POST /payments/123?notify=true
```

```
{
  "amount": 5000
}
```

```
@PostMapping("/payments/{paymentId}")
public PaymentResponse updatePayment(
    @PathVariable String paymentId,
    @RequestParam boolean notify,
    @RequestBody PaymentRequest request) {
    ...
}
```

Here:

```
paymentId → @PathVariable
notify     → @RequestParam
JSON      → @RequestBody
```

6. Important interview distinction

Path variable

```
GET /payments/123
```

means:

Give me **payment 123**.

Request parameter

```
GET /payments?status=SUCCESS
```

means:

Give me payments **filtered by status**.

That distinction is very important in REST API design.

7. Real BPS example

Imagine you have:

```
POST /api/v1/payments
```

Request:

```
{
  "customerId": "C101",
  "amount": 5000,
  "paymentMode": "CASH"
}
```

```
@PostMapping("/payments")
public PaymentResponse createPayment(
    @Valid @RequestBody PaymentRequest request) {
    return paymentService.createPayment(request);
}
```

Then retrieving it:

```
GET /api/v1/payments/TXN1001
```

```
@GetMapping("/payments/{transactionId}")
public PaymentResponse getPayment(
    @PathVariable String transactionId) {
    return paymentService.getPayment(transactionId);
}
```

Then searching:

```
GET /api/v1/payments?status=SUCCESS&page=0&size=20
```

```
@GetMapping("/payments")
public Page<PaymentResponse> getPayments(
    @RequestParam String status,
    @RequestParam int page,
    @RequestParam int size) {

    return paymentService.getPayments(status, page, size);
}
```

8. Common interview questions

Q1. What is `@RequestBody`?

It binds the HTTP request body to a Java object. In a JSON REST API, Spring/Jackson typically deserializes the JSON into that object.

Q2. What is `@PathVariable`?

It extracts a value embedded in the URI path and is commonly used to identify a specific resource.

Q3. What is `@RequestParam`?

It extracts query parameters from the URL and is commonly used for filtering, searching, sorting, pagination, and optional parameters.

Q4. Can `@RequestBody` and `@PathVariable` be used together?

Yes.

```
@PutMapping("/payments/{id}")
public PaymentResponse update(
    @PathVariable Long id,
    @RequestBody PaymentRequest request) {
    ...
}
```

Q5. Can you have multiple `@RequestParam`s?

Yes.

```
@RequestParam String status,
@RequestParam String currency
```

Q6. What happens if a required `@RequestParam` is missing?

Spring generally returns **400 Bad Request** because the required request parameter could not be resolved.

Q7. What happens if the JSON in `@RequestBody` is invalid?

Spring typically fails during JSON deserialization and returns a **400 Bad Request**, subject to your application's exception-handling configuration.

9. One level deeper — 3-year interview

You should also know that these annotations are processed as part of Spring MVC's **argument resolution**.

Conceptually:

```
HTTP Request
↓
DispatcherServlet
↓
Handler Mapping
↓
Controller method
↓
Argument Resolution
├── @PathVariable
├── @RequestParam
└── @RequestBody
↓
Business logic
```

For `@RequestBody`, Spring uses an appropriate `HttpMessageConverter`; for JSON, this is commonly Jackson.

This is the kind of follow-up an interviewer can ask after the basic definition.

Interview-ready summary

Remember this:

```
@RequestBody
→ Data inside HTTP body
→ JSON/XML/etc.
→ Create/update structured object

@PathVariable
→ Data inside URL path
→ Identifies a specific resource

@RequestParam
→ Data after ?
→ Filtering/search/pagination/options
```

A very good mental example is:

```
POST /payments/123?notify=true
```

```
{
  "amount": 1000
}
```

```
123           → @PathVariable
notify=true   → @RequestParam
JSON          → @RequestBody
```

For your next preparation topic, **ResponseEntity**, **HTTP status codes**, **DTOs**, **validation (@Valid)** and **global exception handling** should come immediately after these because interviewers often connect all of them in one REST API question.